

API BOLA & BFLA — Interview Prep Notes

Quick Differentiator

	BOLA (API1:2023)	BFLA (API5:2023)
Checks	Object-level access (data)	Function-level access (action/endpoint)
Question	"Can I access this record ?"	"Can I call this function ?"
Auth check missing on	Object ID ownership	Endpoint/role/privilege
Example	<code>GET /orders/{id}</code> — victim's order	<code>POST /admin/users/delete</code> — low-priv user hits admin endpoint
CWE overlap	Same family (CWE-639/862/863)	Same family, function scope

Both = missing/broken **authorization**, not authentication. Answering combined below; BFLA deltas marked.

1. Logical Description

BOLA: API returns/mutates object identified by user-supplied ID without verifying requester owns/is-permitted that object → horizontal privilege escalation via object reference. **BFLA:** API exposes a function/endpoint (often admin/privileged) reachable by a role that shouldn't have access → vertical privilege escalation via function reference.

2. OWASP Category & CWE

- **API1:2023 – Broken Object Level Authorization** (was API1:2019 too)
- **API5:2023 – Broken Function Level Authorization**
- CWE-639 (Authorization Bypass Through User-Controlled Key) — primary for BOLA
- CWE-862 (Missing Authorization)
- CWE-863 (Incorrect Authorization)
- CWE-284 (Improper Access Control) — parent
- Web app equivalent: OWASP Top10 2021 **A01 Broken Access Control** (IDOR is BOLA's web-app cousin)

3. Common Variants

BOLA:

- Sequential/numeric ID enumeration
- UUID/GUID leaked elsewhere (response body, JWT, another endpoint) → reused
- Nested object BOLA (`/users/{id}/invoices/{invoiceId}` — invoiceId not scoped to userId)
- Batch/bulk endpoint BOLA (array of IDs, only first validated)
- Indirect BOLA via export/report/webhook features
- BOLA in GraphQL (`node(id:)` global ID resolvers)

BFLA:

- Horizontal→vertical: regular user calling admin CRUD endpoint
- HTTP verb tampering (`GET` allowed, `DELETE` on same route not checked)
- Deprecated/shadow API version exposing old unrestricted function
- Client-side only restriction (UI hides button, API still open)
- Role confusion in JWT/claims not re-validated server-side

4. Attack Surface / Entry Points

- ☐ REST path params: `/api/v1/resource/{id}`
- ☐ Query params: `?user_id=`, `?account=`
- ☐ Request body IDs (PUT/PATCH/POST JSON)
- ☐ Headers (`X-User-Id`, custom tenant headers)
- ☐ JWT/session claims vs actual enforced check
- ☐ GraphQL `id` / `node` args, batched queries
- ☐ WebSocket message payload IDs
- ☐ File download/export endpoints (`/reports/{id}.pdf`)
- ☐ Admin/internal endpoints exposed on same base path (BFLA)
- ☐ Mobile app hidden/undocumented endpoints (reverse-engineered APK)
- ☐ API gateway routes bypassing service-level authz (shadow/legacy versions)

5. Testing Workflow (Manual / Burp / ZAP / Nuclei — unified)

Step 1 — Map attack surface

- Verify: full endpoint inventory + object-owning params identified
- Manual: crawl app as 2 users (A, B), catalog every ID-bearing request
- Burp: spider + Logger, use Burp's built-in "Authorization" auditing; Match & Replace to tag requests
- ZAP: Spider/AJAX Spider, HUD to inspect
- Nuclei: N/A at this stage (recon only — feed into your gau/waybackurls/Katana → SQLite inventory)

Step 2 — Create two accounts, two privilege tiers

- Verify: role/tenant separation exists in test data
- Manual: register UserA, UserB (same role) + UserAdmin (BFLA)
- Burp: store both sessions, use **Repeater** with saved auth tabs
- ZAP: use Context + separate session tokens per user
- Nuclei: N/A

Step 3 — Object substitution test (BOLA)

- Verify: UserA's token can access UserB's object
- Manual: capture UserA request → replace object ID with UserB's ID → resend with UserA's token
- Burp: **Repeater** swap ID + auth header; **Authorize/Auth Analyzer extension** for automated matrix (send as UserA, replay with UserB perms, diff response)
- ZAP: **Access Control Testing add-on** (compares authenticated vs unauthenticated/other-role responses)
- Nuclei: custom template with `{{id}}` fuzz + two auth headers, compare status/body — no strong existing template (too app-specific), write custom
- Payloads: sequential ints (± 1 , 0, negative), other user's known UUID, `null`, `0`, wildcard `*`, array injection `["id1", "id2"]`

Step 4 — Function substitution test (BFLA)

- Verify: low-priv token can call high-priv endpoint/method
- Manual: take documented admin endpoint, replay with low-priv token; also try verb tampering (`GET` → `DELETE` / `PUT` same URL)
- Burp: **Repeater** change method via right-click "Change request method"; Authorize for role-matrix automation
- ZAP: Access Control add-on with role definitions
- Nuclei: custom template hitting known admin paths (`/admin/*`, `/internal/*`) with low-priv token, assert 200 instead of 401/403
- Payloads: verb tampering, path case variation (`/Admin/`), trailing slash, `../` path bypass, alternate content-type

Step 5 — Response diffing / behavioral confirmation

- Verify: response contains victim's real data (not empty/error shaped like success)
- Manual: diff response body length, key fields (PII), status code semantics (200 vs 403 with 200-shaped error page = false 403)
- Burp: Comparer (diff baseline vs test response)
- ZAP: manual diff via History panel
- Nuclei: `matchers` on status + body regex for PII markers

Step 6 — Blind/indirect BOLA (no visible ID leak)

- Verify: enumeration via side channel (timing, error msg difference, IDOR in export/webhook)
- Manual: trigger export/report feature with foreign ID, check delivered file
- Burp: Collaborator for OOB confirmation if async (webhook delivers to attacker-controlled URL)

- ZAP: OAST via ZAP's own callback or manual
- Nuclei: interactsh-based template if OOB applicable

6. Evidence to Collect

- Request/response pair: UserA token + UserB's object ID → 200 + UserB's actual data
- Side-by-side screenshot/diff of unauthorized vs authorized response
- Confirmation object belongs to victim (unique PII/order#/email matching victim account)
- Session/token used (redact after capture) + timestamp
- For BFLA: proof action executed (e.g., record actually modified/deleted — verify via victim account after)
- HTTP history export (Burp/ZAP) as raw evidence

7. Good PoC Looks Like

1. Two labeled test accounts (Attacker, Victim) with distinct identifiable data
2. Raw HTTP request as Attacker referencing Victim's object ID
3. Raw HTTP response showing Victim's private data returned (redact real PII in report, mark placeholders)
4. For BFLA: request + response proving privileged action executed + secondary check (re-fetch as Victim/Admin showing state changed)
5. Step-by-step repro (curl or Burp request file) any triager can replay
6. Business-impact framing sentence ("any authenticated user can enumerate and read all customer orders")

8. Attack Chains

- BOLA → mass data exfiltration (enumerate all IDs → full DB dump)
- BOLA → account takeover (read password-reset token/object belonging to victim)
- BFLA → privilege escalation → full admin takeover → RCE (if admin function allows config/code upload)
- BOLA + BFLA chained: low-priv user reads admin object IDs (BOLA) then calls admin function on them (BFLA)
- BOLA in export → SSRF/webhook exfil to attacker infra
- IDOR/BOLA + Mass Assignment → modify object fields not meant to be attacker-controlled (e.g., `isAdmin:true`)

9. Common False Positives

- 403/401 returned but with 200-status-coded error body (misread as blocked when actually allowed, or vice versa)
- Object doesn't exist (404) mistaken for "properly blocked" — need existing victim object to confirm
- Shared/public object legitimately accessible by design (public listings)
- Caching artifacts (CDN/reverse proxy serving stale cached victim response to any requester — real vuln but root cause is caching, not authz)
- Test/staging accounts with intentionally shared access
- Rate-limited response masking as authz block

10. Prerequisites for Exploitability

- Predictable/guessable/enumerable/leaked object identifiers
- Server-side check missing/incorrect (ownership not tied to session)
- Attacker holds any valid authenticated session (even low-priv)
- For BFLA: endpoint reachable/routable (not network-isolated) + no separate function-level gate

11. What Attacker Achieves

- Read/modify/delete other users' data (BOLA)
- Execute privileged actions without privilege (BFLA)
- Full account takeover, data exfiltration at scale, fraud (financial txn manipulation), service disruption (mass delete)

12. CIA Impact (per variant)

Variant	C	I	A	Example CVSS Vector (base)
BOLA – Read other's object	High	None	None	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
BOLA – Write/update other's object	Low/None	High	None	AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:H/A:N
BOLA – Delete other's object	None	None	High	AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:H
BOLA – Bulk/mass enumeration	High	High	Med	AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L
BFLA – Low priv → admin read	High	None	None	AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N
BFLA – Low priv → admin write/delete	High	High	High	AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H

13. Typical CVSS Severity/Vector

- BOLA single-record read: **Medium-High (5.3–7.5)** – e.g. CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N = 6.5
- BOLA with PII/PCI/PHI bulk exposure: **High-Critical (8.1–9.8)** – Scope Changed if crosses trust boundary/tenant
- BFLA to admin: **Critical (9.0–9.8)** – CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H

14. Business Impact (Financial App Context)

- Cross-account balance/transaction/statement exposure → regulatory breach (RBI/PCI-DSS/GDPR)
- BFLA on admin fund-transfer/approval endpoints → direct fraud, unauthorized transactions
- Mass BOLA enumeration = reportable data breach (PII + financial records) → mandatory disclosure, fines
- Reputational damage, customer trust loss, audit findings, potential license/compliance action
- PCI-DSS: cardholder data exposure via BOLA = scope violation, could trigger revalidation
- Class-action/regulatory exposure scales with number of accounts enumerable (systemic, not single-user)

15. Indicators/Signals

- Sequential/incrementing IDs in URLs or bodies
- Object ID present in JWT but not cross-checked against path/body ID server-side
- Endpoints work identically regardless of which user's token is sent
- Admin routes reachable without role check (200 instead of 403) for standard user

- API docs (Swagger/OpenAPI) exposing internal/admin routes publicly
- 200 OK + full object even when Authorization header swapped between users during recon

16. Root Cause (Code-Level)

BOLA — vulnerable pattern (pseudocode):

```
GET /orders/{orderId}

order = db.getOrder(orderId)    // no ownership filter

return order
```

Missing: `WHERE order.user_id = session.user_id`

BFLA — vulnerable pattern:

```
POST /admin/users/{id}/delete

if (session.isAuthenticated):    // only checks auth, not role

    db.deleteUser(id)
```

Missing: `if session.role != 'admin': deny`

Common schema/design root causes:

- Authorization logic duplicated per-endpoint instead of centralized middleware → inconsistent enforcement
- Trusting client-supplied role/ID claims without server-side re-validation
- ORM/query built directly from path param without scoping to authenticated principal
- Route registered without middleware/decorator (missed during rapid API expansion)

17. Remediation — Secure Coding & Design

- Enforce **object-level** check server-side every request: `resource.owner_id == session.user_id` (or tenant/team scoping)
- Enforce **function-level** RBAC/ABAC centrally — deny-by-default middleware, not per-controller opt-in
- Use indirect object references (map internal IDs to per-user opaque tokens) where feasible
- Centralize authz in framework middleware/policy engine (e.g., OPA, CASL, Spring Security `@PreAuthorize`)
- Re-validate role/claims server-side on every request, never trust JWT claims blindly for sensitive ops without fresh check against source of truth
- Deny-by-default route registration; explicit allow-list for public endpoints
- Automated authz tests in CI (role-matrix test per endpoint)

Language-agnostic check:

```
authorize(principal, action, resource):  
    if resource.tenant_id != principal.tenant_id: deny  
    if action not in principal.role.permissions: deny  
    allow
```

18. Compensating Controls (if root cause fix delayed)

- WAF/API gateway rule blocking cross-ID access patterns (heuristic, imperfect)
- Rate limiting + anomaly detection on ID enumeration patterns
- Logging/alerting on 403/404 spikes per user (enumeration attempts)
- Temporarily disable/feature-flag risky bulk/export endpoints
- Add API gateway-level route allow-list restricting admin paths by network/IP/VPN
- Increase ID entropy (switch sequential → UUID) as stopgap — reduces enumeration, doesn't fix authz

19. If Seen Repeatedly — Posture Signal

- No centralized authorization framework/library — each team reimplements ad hoc → systemic
- Authz not covered in SDLC threat modeling/code review checklist
- No automated authz regression tests → same class reintroduced with each new endpoint
- API sprawl outpacing security review (shadow APIs, undocumented routes)
- Indicates need for: shared authz middleware/library, mandatory security review gate for new endpoints, SAST rule enforcement

20. Upstream Improvements

- **SAST/Semgrep taint rule:** flag DB query using path/body param as lookup key without accompanying ownership filter in same function
- **CodeQL:** dataflow from `request.params` → ORM query sink without intermediate authz check node
- Framework-level guard: mandatory `@RequireOwnership` / `@RequireRole` decorator, lint rule fails build if route missing decorator
- OpenAPI spec lint: every path must declare `security` + custom `x-authz-scope` extension; CI fails if missing
- Central authz library (shared package) — single source of truth, versioned, mandated via SAST/dependency check
- Design checklist item: "object ownership check" + "role/function gate" mandatory in API design review template

21. Sample Triage Response

Confirmed BOLA on `GET /api/v1/orders/{id}`. Authenticated low-privilege user (Account A) able to retrieve full order details (PII: name, address, payment last4) belonging to Account B by substituting `orderId` in path. No ownership check server-side. Reproducible 100%, no rate limiting encountered. Severity: High (CVSS 7.5, confirmed bulk enumeration possible → escalating to Critical/9.1 pending scope discussion). Recommend immediate hotfix (ownership filter) + gateway rate-limit as interim control. Evidence attached (Repeater request/response pair).

22. Sample Developer Remediation Note

Issue: `/orders/{id}` endpoint fetches order by ID without verifying `order.user_id == req.user.id`. **Fix:** Add ownership filter in query: `SELECT * FROM orders WHERE id=? AND user_id=?`. Reject with 403 (not 404, to avoid behavior leak — but see note on enumeration vs info-leak tradeoff) if mismatch. **Regression test:** Add authz test case — Account A token + Account B's order ID → expect 403. **Broader fix:** Audit all endpoints using this ORM pattern for same missing filter (systemic — see IDOR sweep findings). Consider shared `scopedQuery()` wrapper enforcing tenant/owner filter by default.

23. Real-World Examples

- **USPS Informed Delivery/Account API (2018)** — BOLA allowed access to any user's mail-tracking/account data via account number manipulation
- **Facebook Instagram BOLA (bug bounty, ~2020)** — object reference on private story/analytics endpoints exposed other users' data
- **Peloton API BOLA (2021)** — unauthenticated/low-priv requests exposed private user data (age, location, workout stats) via user ID in API calls, no ownership check
- **T-Mobile API BOLA (2023 breach)** — attacker enumerated customer account API to pull PII for ~37M accounts
- **BFLA class:** multiple fintech HackerOne reports — low-priv user token accepted on `/admin/*` fund-approval endpoints due to missing role middleware (pattern recurring across programs, not single CVE)